

Repeated evaluations of linear transformations

Moses Christian and Ching-Lueh Chang

Department of Computer Science and Engineering

Yuan-Ze University, Jhongli City, Taoyuan County 320, Taiwan

s1016008@mail.yzu.edu.tw

Abstract

This paper deals with the optimization of repeated evaluations of linear transformations. We focus on the situation that different sequence of additions and substitutions would take different time to apply linear transformations. Our research contains analysis of linear transformations and a greedy algorithm is used to find the optimal solution.

1 Introduction

Fast evaluation of a composition of linear transformations has been an active research topic. A classic result, for example, is a polynomial-time algorithm for parenthesizing a product of matrices to minimize the number of arithmetic operations needed to evaluate the product.

In evaluating a linear transformation on a linear combination of vectors, we usually apply addition first and then substitute the sum into the function. For each field F , a linear transformation from F^n to F^m could be represented by an m by n matrix. For example, suppose that we have four matrices A, B, C and D where each represents a linear transformation. $ABCD$ is the composition of the linear transformations and equals $((AB)C)D, ((A(BC))D), (AB)(CD), A((BC)D)$ and $A(B(CD))$ by the associativity of matrix multiplication. The goal is to find an arrangement taking the minimum time to compute [1]. Here we have some linear transformations $f_i: V_i \rightarrow V_{i+1}, i \in \{1, 2, \dots, k-1\}$, where $V_1, V_2, \dots, V_k$ are vector spaces over a field F . Given

$$\begin{aligned} c_1, c_2, \dots, c_n &\in F, \\ x_1, x_2, \dots, x_n &\in V_1, \\ \text{there are many ways to calculate} \\ f_{k-1} \left(\dots \left(f_2 \left(f_1 \left(\sum_{i=1}^n c_i x_i \right) \right) \right) \dots \right). \end{aligned} \quad (1.1)$$

Assume that in V_i , a scalar multiplication costs s_i , vector addition costs t_i and

calculating f_i costs u_i . Here we have a problem:

Given s_i, t_i, u_i for all $i \in \{1, 2, \dots, k-1\}$, find

a way to calculate equation (1.2) with the lowest cost.

To get into the problem, we classify the vectors into two main classes. The vectors in the form of $f_{i-1} \left(f_{i-2} \left(\dots f_1 \left(\sum c_j x_j \right) \dots \right) \right)$ are classified in A_i of size a_i , and those in the form of $f_{i-1} \left(f_{i-2} \left(\dots f_1(x_j) \dots \right) \right)$ are classified in B_i of size b_i . Classes A_i and B_i are subsets of the vector space V_i .

At the beginning, $i = 1$, $A_i = \emptyset$ and $B_i = V_i$. Then, in the i th (where $i > 1$) round, multiply $0 \leq e_i \leq b_i$ vectors in class B_i with their coefficients. It takes e_i multiplications with cost $e_i s_i$ and the number of vectors is increased by e_i in class A_i and decreased by e_i in class B_i . Then take $1 \leq d_i \leq a_i + e_i$ vectors in class A_i for addition. It takes $d_i - 1$ additions with cost $(d_i - 1)t_i$ and decreases $|A_i|$ by $d_i - 1$. At the end of the round, substitute all vectors in classes A_i and B_i into f_i when $i < k$ and no substitution applied when $i = k$.

The i th round takes $d_i - 1$ additions and the total number of additions of the n vectors is $n - 1$, because exactly one vector shall remain in V_k at the end. Thus, we formulate the problem into the following minimization problem:

Minimize

$$\sum_{i=1}^k e_i s_i + (d_i - 1)t_i + \sum_{i=1}^{k-1} \left(n - \sum_{j=1}^i (d_j - 1) \right) u_i \quad (1.2)$$

subject to $1 \leq d_i \leq a_i + e_i$ for all $i \in$

$\{1, 2, \dots, k\}$, $d_i \in \mathbb{Z}$, $0 \leq e_i \leq b_i$, and

$$\sum_{i=1}^k (d_i - 1) = n - 1.$$

We consider a special case of the problem by setting c_i for all $i \in \{1, 2, \dots, n\}$ to the multiplicative identity of the field F so that we can ignore c_i and consider e_i to be zero.

Thus, we have a new minimization problem:

Minimize

$$\sum_{i=1}^k (d_i - 1)t_i + \sum_{i=1}^{k-1} \left(n - \sum_{j=1}^i (d_j - 1) \right) u_i \quad (1.3)$$

subject to $1 \leq d_i \leq a_i$ for all

$i \in \{1, 2, \dots, k\}$, $d_i \in \mathbb{Z}$, and $\sum_{i=1}^k (d_i - 1) =$

$n - 1$.

We solve the new minimization problem using a polynomial time greedy algorithm. We will prove that the greedy algorithm is optimal. It may also be possible to use another approximation method that can solve many approximation problems: formulate the problem into an integer linear programming (ILP) problem, relaxing the problem into a linear programming (LP) problem so that it can be solved in polynomial time [2], rounding the result into a solution to the ILP problem, and then analyzing the approximation ratio. But we will focus on the greedy algorithm in the following sections.

Repeated evaluation is a commonly used method in many cases, such as gradient descent method, Newton's method, and finding the fixed point of a contradiction mapping using iterations of functions [3]. While our research is about the method that is quite common in the modern day, we have not seen any related work about repeatedly evaluating linear transformations by now; to the best of our knowledge, this may be the first work.

2 Definitions

Define the cost function:

$$\text{cost}(d_1, d_2, \dots, d_k) \stackrel{\text{def}}{=} \sum_{i=1}^k (d_i - 1)t_i + \sum_{i=1}^{k-1} \left(n - \sum_{j=1}^i (d_j - 1) \right) u_i \quad (2.1)$$

For $p, q \in \{1, 2, \dots, k-1\}$,

$$\chi[p < q] \stackrel{\text{def}}{=} \begin{cases} 1, & \text{if } p < q, \\ 0, & \text{otherwise.} \end{cases} \quad (2.2)$$

Denote the standard basis of $\mathbb{R}^{k-1}$ by $\{e_1, e_2, \dots, e_{k-1}\}$. So, when we increase d_i by 1 and simultaneously decrease d_j by 1, the cost changes by

$$\text{change}(p, q, d_1, d_2, \dots, d_k) \quad (2.3)$$

$$\stackrel{\text{def}}{=} \text{cost}((d_1, d_2, \dots, d_k) + e_p - e_q)$$

$$- \text{cost}(d_1, d_2, \dots, d_k),$$

assuming $d_q \geq 1$.

3 The proof and analysis

Lemma 1. Let $d_1, d_2, \dots, d_k$ be a feasible solution for Problem (1.3). Then for distinct $p, q \in \{1, 2, \dots, k-1\}$ with $d_q \geq 1$,

$$\begin{aligned} & \text{change}(p, q, d_1, d_2, \dots, d_k) \\ &= t_p - t_q + (-1)^{\chi[p < q]} \sum_{h=\min\{p,q\}}^{\max\{p,q\}-1} u_h. \end{aligned}$$

In particular, $\text{change}(p, q, d_1, d_2, \dots, d_k)$ does not depend on $d_1, d_2, \dots, d_k$.

Proof: For $i, p \in \{1, 2, \dots, k-1\}$,

$$\kappa[i = p] \stackrel{\text{def}}{=} \begin{cases} 1, & \text{if } i = p, \\ 0, & \text{otherwise.} \end{cases} \quad (3.1)$$

We have

$$\begin{aligned} & \text{change}(p, q, d_1, d_2, \dots, d_k) \\ &= \text{cost}((d_1, d_2, \dots, d_k) + e_p - e_q) \\ & \quad - \text{cost}(d_1, d_2, \dots, d_k) \end{aligned}$$

$$\begin{aligned} &= \sum_{i=1}^k (d_i + \kappa[i = p] - \kappa[i = q] - 1)t_i \\ &+ \sum_{i=1}^{k-1} \left(n - \sum_{j=1}^i (d_j + \kappa[j = p] - \kappa[j = q] - 1) \right) u_i \\ &- \left(\sum_{i=1}^k (d_i - 1)t_i + \sum_{i=1}^{k-1} \left(n - \sum_{j=1}^i (d_j - 1) \right) u_i \right) \\ &= \sum_{i=1}^k (d_i - 1)t_i + \sum_{i=1}^k (\kappa[j = p] - \kappa[j = q])t_i \\ &+ \sum_{i=1}^{k-1} \left(n - \sum_{j=1}^i (d_j - 1) \right) u_i \\ &- \sum_{i=1}^{k-1} \left(\sum_{j=1}^i (\kappa[j = p] - \kappa[j = q]) \right) u_i \\ &- \sum_{i=1}^k (d_i - 1)t_i - \sum_{i=1}^{k-1} \left(n - \sum_{j=1}^i (d_j - 1) \right) u_i \\ &= \sum_{i=1}^k (\kappa[i = p] - \kappa[i = q])t_i \\ &- \sum_{i=1}^{k-1} \left(\sum_{j=1}^i (\kappa[j = p] - \kappa[j = q]) \right) u_i \end{aligned}$$

$$= t_p - t_q - \sum_{i=1}^{k-1} \left(\sum_{j=1}^i \kappa[j=p] - \sum_{j=1}^i \kappa[j=q] \right) u_i.$$

Above, the first and second equalities follow from equations (2.3) and (2.1), respectively.

We have

$$\begin{aligned} & \text{change}(p, q, d_1, d_2, \dots, d_k) \\ &= t_p - t_q - \sum_{i=1}^{k-1} (\chi[i \geq p] - \chi[i \geq q]) u_i \end{aligned}$$

since $\sum_{j=1}^i \kappa[j=p] = \chi[i \geq p]$, and $\sum_{j=1}^i \kappa[j=q] = \chi[i \geq q]$ by Definitions (2.2) and (3.1). This gives

$$\begin{aligned} & \text{change}(p, q, d_1, d_2, \dots, d_k) \\ &= t_p - t_q + (-1)^{\chi[p < q]} \sum_{h=\min\{p,q\}}^{\max\{p,q\}-1} u_h \end{aligned}$$

where $(-1)^{\chi[p < q]}$ indicates that if $p < q$ then $\chi[i \geq p] > \chi[i \geq q]$ for all $i \in \{p, p+1, \dots, q-1\}$ and the summation on the right-hand side needs to keep negative, otherwise, it is positive. ■

In light of Lemma 1, we will write $\text{change}(p, q, d_1, d_2, \dots, d_k)$ simply as $\text{change}(p, q)$ for all $d_1, d_2, \dots, d_k$.

Lemma 2. For all $h \in \mathbb{Z}^+$ and $(p(1), q(1)), (p(2), q(2)), \dots, (p(h), q(h)) \in \{1, 2, \dots, k\}^2$ such that all the components of

$$(d_1, d_2, \dots, d_k) - \sum_{s=1}^h e_{q(s)} \quad (3.2)$$

are non-negative,

$$\begin{aligned} & \text{cost} \left((d_1, d_2, \dots, d_k) + \sum_{s=1}^h e_{p(s)} - e_{q(s)} \right) \\ & \quad - \text{cost}(d_1, d_2, \dots, d_k) \\ &= \sum_{s=1}^h \text{change}(p(s), q(s)). \end{aligned}$$

Proof: Clearly,

$$\begin{aligned} & \text{cost} \left((d_1, d_2, \dots, d_k) + \sum_{s=1}^h e_{p(s)} - e_{q(s)} \right) \\ & \quad - \text{cost}(d_1, d_2, \dots, d_k) \\ &= \sum_{i=1}^h \text{cost} \left((d_1, d_2, \dots, d_k) + \sum_{s=1}^i e_{p(s)} - e_{q(s)} \right) \\ & \quad - \text{cost} \left((d_1, d_2, \dots, d_k) + \sum_{s=1}^{i-1} e_{p(s)} - e_{q(s)} \right). \end{aligned}$$

By Lemma 1,

$$\begin{aligned} & \sum_{i=1}^h \text{cost} \left((d_1, d_2, \dots, d_k) + \sum_{s=1}^i e_{p(s)} - e_{q(s)} \right) \\ & \quad - \text{cost} \left((d_1, d_2, \dots, d_k) + \sum_{s=1}^{i-1} e_{p(s)} - e_{q(s)} \right) \\ &= \sum_{i=1}^h \text{change}(p(i), q(i)). \quad \blacksquare \end{aligned}$$

For all $d'_1, d'_2, \dots, d'_{k-1} \in \mathbb{N}$ with $\sum_{i=1}^k d'_i = n-1$, it is easy to find $(p(1), q(1)), (p(2), q(2)), \dots, (p(h), q(h)) \in \{1, 2, \dots, k\}^2$ satisfying inequality (3.2) and

$$(d_1, d_2, \dots, d_k) + \sum_{s=1}^h e_{p(s)} - e_{q(s)} = (d'_1, d'_2, \dots, d'_k) \quad (3.3)$$

We are now able to verify that the following local-search algorithm is optimal.

- 1: Take any $d_1, d_2, \dots, d_k \in \mathbb{N}$ with $\sum_{i=1}^k d_i = n-1$.
- 2: while there exist $i, j \in \{1, 2, \dots, k\}$ with $d_j \geq 1$ and $\text{change}(i, j) < 0$ do
- 3: $d_i \leftarrow d_i + 1$.
- 4: $d_j \leftarrow d_j - 1$.
- 5: end while

4 Conclusion

In this paper, we have shown that the minimization problem of linear transformations can be solved using the greedy algorithm and scalar multiplications are ignored to reduce the difficulty. Currently, we are still working on the issues of considering the multiplications in the linear transformations and developing a solution to the problem.

References

- [1] Thomas H. Cormen, Charles E. Leiserson and Ronald L. Rivest, Clifford Stein, *Introduction to Algorithms*, 3rd Edition The MIT Press, 2009.
- [2] L. G. Kachiyan (1979), A Polynomial Algorithm in Linear Programming. *Soviet Math. Dokl.* 20, 191–194.
- [3] G. Dahlquist and A. Björck, *Numerical Methods*, Dover Publications, 2003.